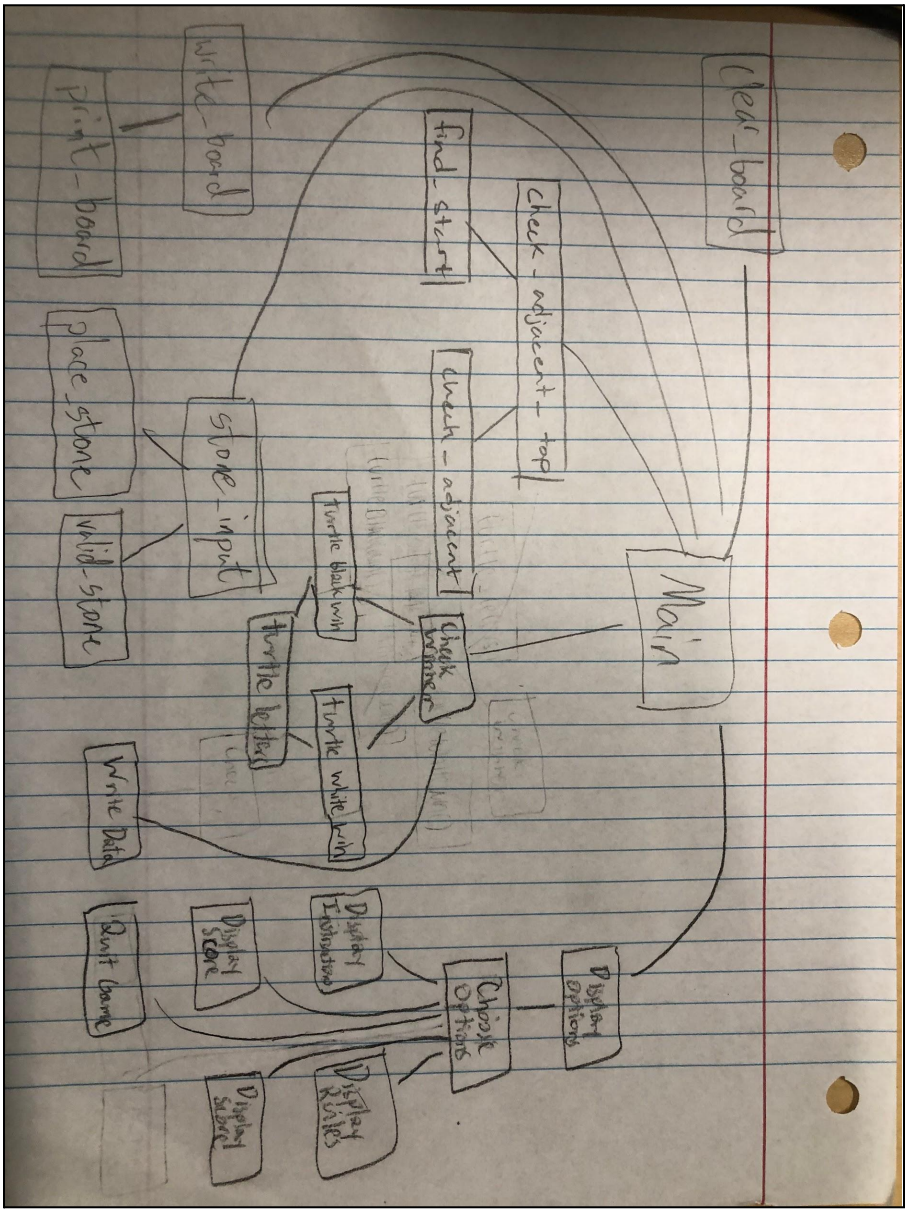
Group Members: Vince Young, Basel Haque, Ethan Miller, Grant Perry

Description:
We determined that Pente was best suited for our group due to our familiarity with the lists of lists from the go board assignment and the simplicity of how the individual turns should be coded. Knowing how to make and modify the mock go board is essential in attempting to make a functional and playable game, so having a firm basis to work from makes this step significantly easier. Reading the rules made it clear that a capture check only happens when a tile is placed, and not retroactively. This makes the order of operations for the game linear and without a distinct endgame like Teeko. Essentially, we decided a longer set of rules was easier to manage rather than two different sets of rules.

Hierarchy:



User Manual

When you start the program, a turtle graphics window will pop up. Simply click away and you will be able to run the program. After clicking away from the turtle graphics window, you are prompted to input a set of numbers or the letter ‘o’. The numbers must be inputted in the format <Num1>,<Num2> with no spaces. For example: ‘12,3’. The left number is the row number whilst the right number is the column number you want to place your stone.

When a player wins, they will be able to see the appropriate win screen drawn using turtle graphics. Furthermore, a text file will be created after program ends that allows the users to see a history of any games played during their current session; this file shows the game number, which side won, and how many captures each side made.

‘O’ are white stones.
‘X’ are black stones.

If you enter ‘o’, you are given a list of options:

1. Instructions
2. Game Rules
3. Score
4. Quit Game

Enter the number corresponding to the option to see the contents of that specific option.

- The ‘instructions’ option gives a brief description on how to run the program.
- The ‘Game Rules’ option gives you the rules of the game you’re playing, Pente.
- The ‘Score’ option gives you the total amount of wins black and white have accumulated.
- The ‘Quit Game’ option allows you to stop the program.

Proper input and output examples:

1. Placing a stone:

```
White, enter your row and column number separated by a comma to place a stone,
or enter 'o' to open up options: 1,1

   1  2  3  4  5  6  7  8  9  10 11 12 13 14 15 16 17 18 19
1  O  -  -  -  -  -  -  -  -  -  -  -  -  -  -  -  -  -  -
2  -  -  -  -  -  -  -  -  -  -  -  -  -  -  -  -  -  -  -
```

(Not the full output)

2. Opening up the options menu.

```
White, enter your row and column number separated by a comma to place a stone,
or enter 'o' to open up options (Includes rules and instructions!): o

PROGRAM OPTIONS
---------------
1. Instructions
2. Game Rules
3. Score
4. Quit Game
---------------
Enter the appropriate number to access each option:
```

How the code works:

We created a multitude of functions to operate individual game functions using a mixed approach, calling functions within functions and primarily in main. The main code runs indefinitely until quit so players may play as many games as desired, and each win is displayed and documented by a file. The file records game number, whole won, and number of individual captures. Each of these stats are recorded within functions and documented when appropriate. Functions are run in a loop (main) to play games consecutively. Recursion as a concept, calling a function within a function, was used multiple times as the concept not taught in class to expedite the main code and simplify the overall code.

Ethan's Functions:

- check_adjacent(start_pos):
  - Checks all surrounding positions on the board to see if there's potential for a capture or 5 in a row.
- check_adjacent(start_pos, direction, iter, is_capture):
  - Recursive helper method which continues in the initial direction.

Basel's Functions:

- turtleWhiteWin(whiteWin)
  - Creates turtle graphics displaying a victory screen for white
- turtleBlackWin(blackWin)
  - Creates turtle graphics displaying a victory screen for black
- Turtle Graphic Functions for all needed letters
  - Functions to write each individual letter to be called in the win graphics functions

Vince's Functions:

- displayOptions():
  - Displays a list of options (score, rules, instructions, quit early) to the user and prompts user to choose an option
- chooseOption():
  - Takes input from the user and displays the desired option.
- displayRules():
  - Function that prints out all rules for Pente
- displayInstructions():
  - Function that prints out instructions for running the game
- displayScore():
  - Displays the number of wins white and black have
- quitGame():
  - Stops the game completely by breaking the main while loop
- writeData():
  - Writes game number, captures for black and white, and who won into a log file

Grant's Functions:

- write_board1()
  - Creates the initial board, everything is blank
- clear_board()
  - Completely resets board to initial state
- print_board()
  - Prints board in its current state
- stone_input(white)
  - User input loop to take a valid row & column and places stone appropriately, calls "place_stone" and "valid_stone"
- place_stone(row, column, white)
  - Places stone in requested row/column according to player color

- valid_stone(row, column)
    - Checks if the suggested stone can be placed and prompts a “try again” if unable to place stone

```
import sys
import turtle

def B(t):
  t.setheading(0)
  t.penup()  # MOVING PEN TO PROPER LOCATION
  # draw straight line
  t.goto(-300, 300)  # B in Black.   Putting in proper position
  t.pendown()  # PEN BEGINS TO WRITE
  t.pensize(10)  # PEN SIZE
  t.pencolor("green")  # PEN COLOR
  t.right(90)
  t.forward(200)
  t.penup()
  t.goto(-300, 300)
  # draw first curve
  t.pendown()
  t.right(-90)
  t.circle(-50, 180)
  t.penup()
  t.goto(-300, 200)
  # draw second curve
  t.pendown()
  t.right(180)
  t.circle(-50, 180)

def L(t):  # L in Black
  t.setheading(0)
  t.penup()
  # draw straight line
  t.goto(-200, 300)  # putting in proper position
  t.pendown()
  t.pensize(10)
  t.pencolor("purple")
  t.right(90)
  t.forward(150)
  t.right(-90)
  t.forward(70)

def A(t):  # A in Black
  t.setheading(0)
  t.penup()
  t.goto(-150, 300)  # putting in proper position
  t.pendown()
  t.pensize(10)
```

```
    t.pencolor("orange")
    t.right(65)
    t.forward(100)
    t.setpos(-150, 300)
    t.right(50)
    t.forward(100)
    t.penup()
    t.setpos(-170, 240)
    t.right(65)
    t.pendown()
    t.backward(50)

def C(t):  # C in Black
    t.setheading(0)
    t.penup()
    # draw straight line
    t.goto(0, 300)  # putting in proper position
    t.pendown()
    t.pensize(10)
    t.pencolor("blue")
    t.right(180)
    t.circle(50, 180)

def K(t):  # K in Black
    t.setheading(0)
    t.penup()
    # draw straight line
    t.goto(120, 300)  # putting in proper position
    t.pendown()
    t.pensize(10)
    t.pencolor("purple")
    t.right(90)
    t.forward(150)
    t.goto(120, 230)
    t.left(45)
    t.forward(100)
    t.goto(120, 230)
    t.left(90)
    t.forward(100)

def W(t):  # W in Wins
    t.setheading(0)
    t.penup()
    t.goto(-300, 0)  # putting in proper position
    t.pendown()
    t.pensize(10)
    t.pencolor("pink")
    t.right(65)
    t.forward(100)
```

```
    t.left(130)
    t.forward(100)
    t.right(120)
    t.forward(100)
    t.left(130)
    t.forward(100)

def I(t):  # I in Wins
    t.setheading(0)
    t.penup()
    # draw straight line
    t.goto(-100, 0)  # propper position
    t.pendown()
    t.pensize(10)
    t.pencolor("blue")
    t.right(90)
    t.forward(150)

def N(t):  # N IN WINS
    t.penup()
    t.setheading(0)
    # draw straight line
    t.goto(0, 0)  # Putting in proper position
    t.pendown()
    t.pensize(10)
    t.pencolor("red")
    t.right(90)
    t.forward(150)
    t.goto(0, 0)
    t.goto(80, -140)
    t.goto(80, 0)

def S(t):  # S in WINS
    t.penup()
    t.setheading(0)
    # draw straight line
    t.goto(200, 0)  # put in proper position
    t.pendown()
    t.pensize(10)
    t.pencolor("turquoise")
    t.right(180)
    t.circle(50, 180)
    t.left(180)
    t.circle(50, -180)

def W_in_white(t):  # W in White
    t.penup()
    t.setheading(0)
    t.goto(-300, 300)  # put in proper position
```

```
    t.pendown()
    t.pensize(10)
    t.pencolor("orange")
    t.right(85)
    t.forward(150)
    t.left(160)
    t.forward(100)
    t.right(120)
    t.forward(100)
    t.left(145)
    t.forward(150)

def H(t):  # H IN WHITE
    t.penup()
    t.setheading(0)
    # draw straight line
    t.goto(-100, 300)  # PROPER POSITION
    t.pendown()
    t.pensize(10)
    t.pencolor("green")
    t.right(90)
    t.forward(200)
    t.goto(-100, 200)
    t.goto(-20, 200)
    t.goto(-20, 300)
    t.goto(-20, 110)

def I_in_white(t):  # I IN WHITE
    t.penup()
    t.setheading(0)
    # draw straight line
    t.goto(20, 300)  # centering in the screen
    t.pendown()
    t.pensize(10)
    t.pencolor("black")
    t.right(90)
    t.forward(150)




def T(t): # T IN WHITE
    t.penup()
    t.setheading(0)
    t.goto(40, 300)
    t.pendown()
    t.pensize(10)
    t.pencolor("brown")
    t.forward(100)
```

```
    t.goto(90, 300)
    t.right(90)
    t.forward(100)

def E(t):  # E IN WHITE
    t.penup()
    t.setheading(0)
    t.setpos(175, 300)
    t.pendown()
    t.pensize(10)
    t.pencolor("magenta")
    t.forward(50)
    t.backward(50)
    t.right(90)
    t.forward(50)
    t.left(90)
    t.forward(50)
    t.backward(50)
    t.right(90)
    t.forward(50)
    t.left(90)
    t.forward(50)

def exclamation(t):  # EXCLAMATION POINT AT END
    t.penup()
    t.setheading(0)
    t.goto(250, 0)  #
    t.pendown()
    t.pensize(5)
    t.pencolor("yellow")
    t.forward(100)
    t.left(75)
    t.forward(-200)
    t.left(30)
    t.forward(200)
    t.penup()
    t.goto(310, -225)
    t.pendown()
    t.circle(10)

def turtleBlackWin():  # IF BLACK WINS
    turtle.clearscreen()
    t = turtle.Turtle()
    B(t)
    L(t)
    A(t)
    C(t)
    K(t)
    W(t)
```

```
  I(t)
  N(t)
  S(t)
  exclamation(t)

def turtleWhiteWin():  # IF WHITE WINS
  turtle.clearscreen()
  t = turtle.Turtle()
  W_in_white(t)
  H(t)
  I_in_white(t)
  T(t)
  E(t)
  W(t)
  I(t)
  N(t)
  S(t)
  exclamation(t)


gameHistory = open("Game_History.txt", 'w')
numWhiteWins = 0
numBlackWins = 0
numGames = 0
not_quitting = False

def displayOptions(white):
  '''Displays program options that provide useful information to user'''
  print(f"PROGRAM OPTIONS\n"
        f"---------------\n"
        f"1. Instructions\n"
        f"2. Game Rules\n"
        f"3. Score\n"
        f"4. Quit Game\n"
        f"---------------")
  optionInput = input("Enter the appropriate number to access each option: ")
  correctInputs = ['1', '2', '3', '4']
  while True:
      # Check if user input for options is valid
      if optionInput in correctInputs:
          print()
          return chooseOption(optionInput, white)
      else:
          # Prompt user for new input if their input was invalid
          print("\nInvalid input!")
          optionInput = input("Enter the appropriate number to access each
option: ")
```

```
def displayInstructions(white):
  '''Prints out set of instructions'''
  print(f"INSTRUCTIONS\n"
        f"---------------\n"
        f"This program plays the game called 'Pente.'\n"
        f"White stones are denoted by 'O', whilst black stones are denoted by
'X.'\n"
        f"You will be prompted to do a multiple of actions.\n"
        f"The most important action is inputting stones on the board.\n"
        f"You must input 2 integers separated by 1 comma to place a stone
properly.\n"
        f"The first integer provides the row number, whilst the second integer
provides the column number.\n"
        f"Do not use any spaces between the numbers or comma.\n"
        f"A proper input to place a stone on row 1, column 1. would be '1,1'\n"
        f"After a game is won, a victory screen will be drawn. Do not input
anything until it is finished drawing."
        f"Furthermore, a log of games played during your session can be seen in
the created Game_History.txt file after you end your session.")
  return white



def displayRules(white):
  '''Prints out the game's rules'''
  print(f"GAME RULES\n"
        f"---------------\n"
        f"1. White always assumes the opening move.\n"
        f"2. Stones are captured by custodial capture. If a stone is captured,
it is removed.\n"
        f"3. Captures consist of exactly two stones; flanking 1 stone or 3 or
more stones does not result in a capture.\n"
        f"\t- For example, if the stones are X O O _ and 'X' places their stone
so it becomes X O O X,\n"
        f"\t  then the opponent's stones are captured and removed from the
board, leaving X _ _ X.\n"
        f"4. A stone may legally be placed on any empty intersection, even if
it forms a pair between two enemy stones.\n"
        f"\t- If the stones are placed X O _ X , then 'O' may place their stone
so it becomes X O O X.\n"
        f"\t  The stones are not captured in this case.\n"
        f"5. A player may win in 2 ways:\n"
        f"\t- If they make 5 captures.\n"
        f"\t- If they align 5 or more stones of the same color in a row\n"
        f"\t  in any vertical, horizontal or diagonal direction.")
  return white
```

```
def displayScore(white):
  '''Prints out the number of wins for white and black respectively'''
  whiteScore = numWhiteWins
  blackScore = numBlackWins
  print(f"GAME RULES\n"
        f"---------------\n")
  # Check whether score is singular or multiple and prints grammatically
correct sentence.
  if whiteScore == 1:
      print(f"White has {whiteScore} win.")
  else:
      print(f"White has {whiteScore} wins.")
  if blackScore == 1:
      print(f"Black has {blackScore} win.")
  else:
      print(f"Black has {blackScore} wins.")
  return white



def quitGame():
  '''Quits the game and stops the program'''
  print(f"Shutting down game.\n"
        f"Thank you for playing!")
  gameHistory.close()
  sys.exit(1)



def chooseOption(optionInput, white):
  '''Choose option to display based on user input'''
  if optionInput == '1':
      return displayInstructions(white)
  elif optionInput == '2':
      return displayRules(white)
  elif optionInput == '3':
      return displayScore(white)
  elif optionInput == '4':
      return quitGame()
```

```
def writeData(gameNumber, whiteWin, blackCaptures, whiteCaptures):
  if whiteWin:
      gameHistory.write(f"GAME {gameNumber}\n"
                        f"---------------\n"
                        f"Won by White.\n"
                        f"White had captured {str(whiteCaptures)} pairs of
stones.\n"
                        f"Black had captured {str(blackCaptures)} pairs of
stones.\n")
  else:
      gameHistory.write(f"GAME {gameNumber}\n"
                        f"---------------\n"
                        f"Won by Black.\n"
                        f"White had captured {str(whiteCaptures)} pairs of
stones.\n"
                        f"Black had captured {str(blackCaptures)} pairs of
stones.\n")



def check_adjacent_top(pos):
  """"""
  count = 0
  for x in range(-1, 2):
      for y in range(-1, 2):
          if not ((pos[0] == 0 and x == -1) or (pos[0] == 18 and x == 1) or
(pos[1] == 0 and y == -1) or (
                  pos[1] == 18 and y == 1)):
              if '-' not in board[pos[0] + x][pos[1] + y] and (x != 0 or y !=
0):
                  bpos = find_start(pos, [x, y])
                  if bpos != pos:
                      curr = check_adjacent([bpos[0] - x, bpos[1] - y], [-x,
-y], False, 1)
                  else:
                      curr = check_adjacent([bpos[0] + x, bpos[1] + y], [x, y],
True, 1)
                  if curr[1]:
                      return curr
                  count += curr[0]
  return count, False



def check_adjacent(pos, next_p, is_capture, i):
  """Checks both for captures and if there are 5 same colored pieces in a row
through recursion"""
```

```
  if is_capture:
      if not ((pos[0] == 0 and next_p[0] == -1) or (pos[0] == 18 and next_p[0]
== 1) or (
              pos[1] == 0 and next_p[1] == -1) or (pos[1] == 18 and next_p[1]
== 1)):
          if board[pos[0] + next_p[0]][pos[1] + next_p[1]] ==
board[pos[0]][pos[1]]:
              if i < 2:
                  ret = check_adjacent([pos[0] + next_p[0], pos[1] +
next_p[1]], next_p, True, i + 1)
                  if ret[0] == 2:
                      board[pos[0]][pos[1]] = '-  '
                  return ret
              else:
                  return 0, False
          else:
              if i == 2 and '-' not in board[pos[0] + next_p[0]][pos[1] +
next_p[1]]:
                  board[pos[0]][pos[1]] = '-  '
                  return 2, False
      else:
          return 0, False
  elif not ((pos[0] == 0 and next_p[0] == -1) or (pos[0] == 18 and next_p[0] ==
1) or (
          pos[1] == 0 and next_p[1] == -1) or (pos[1] == 18 and next_p[1] ==
1)):
      if board[pos[0] + next_p[0]][pos[1] + next_p[1]] in
board[pos[0]][pos[1]]:
          if i < 4 and not ((pos[0] == 0 and next_p[0] == -1) and (pos[0] == 18
and next_p[0] == 1) and (
                  pos[1] == 0 and next_p[1] == 0) and (pos[1] == 18 and
next_p[1] == 1)):
              return check_adjacent([pos[0] + next_p[0], pos[1] + next_p[1]],
next_p, False, i + 1)
          else:
              return 0, True
      elif i == 4 and board[pos[0] - next_p[0]][pos[1] - next_p[1]] in
board[pos[0]][pos[1]]:
          return 0, True
  return 0, False




def find_start(start, next_p):
  """In case the player placed a stone in the middle of a row, goes backwards
to find best starting position"""
  if not ((start[0] == 0 and next_p[0] == -1) or (start[0] == 18 and next_p[0]
== 1) or (
```

```
          start[1] == 0 and next_p[1] == -1) or (start[1] == 18 and next_p[1]
== 1)):
      if board[start[0]][start[1]] in board[start[0] + next_p[0]][start[1] +
next_p[1]]:
          return find_start([start[0] + next_p[0], start[1] + next_p[1]],
next_p)
  return start




##### GRANT'S CODE ######
board = []




def write_board():
  '''creates initial board'''
  for i in range(19):
      board.append(['-  '] * 19)  # makes a uniform board
  print_board()  # initial board viewing




def clear_board():
  '''completely resets board to a neutral state'''
  for i in range(19):
      for j in range(19):
          board[i][j] = "-  "  # sets all to the initial string
  print_board()  # initial board view




def print_board():
  '''prints board in its current state'''
  row_label = ["1  ", "2  ", "3  ", "4  ", "5  ", "6  ", "7  ", "8  ", "9  ",
"10 ", "11 ", "12 ", "13 ", "14 ",
               "15 ", "16 ", "17 ", "18 ", "19 "]
  print("   ", end="")  # spaced for proper board formatting
  for i in range(len(board)):  # Prints column labels
      if i < 18:
          print(row_label[i], end="")
      else:
          print(row_label[i])  # needs newline for board to look correct
```

```
  for a in range(len(board)):  # a is row number
      print(row_label[a], end="")  # Prints the row labels
      for b in range(len(board[a])):
          print(board[a][b], end='')
      print()



def stone_input(is_white):
  '''takes user input for stone placement, white var determines color'''
  try:
      placeable = False  # for the while loop
      options = False
      if is_white:  # to address current player *
          color = "White"
      else:
          color = "Black"  # *
      while not placeable:
          inputIn = input(f"{color}, enter your row and column number separated
by a comma to place a stone,\n"
                          "or enter 'o' to open up options (Includes rules and
instructions!): ")
          while inputIn == "o":
              print()
              is_white = displayOptions(is_white)
              print()
              options = True
              inputIn = input(f"{color}, enter your row and column number
separated by a comma to place a stone,\n"
                              "or enter 'o' to open up options (Includes rules
and instructions!): ")
          row_inT, column_inT = inputIn.split(',')
          row_in = int(row_inT)
          column_in = int(column_inT)
          print()  # formatting
          placeable = valid_stone(row_in, column_in)
      place_stone(row_in, column_in, is_white)  # places
      return row_in - 1, column_in - 1
  except TypeError:
      if options == True:
          print()
      else:
          print("Invalid input, try again!")  # catch all for invalid inputs
      return stone_input(is_white)
  except IndexError:
      if options == True:
          print()
      else:
```

```
            print("Invalid input, try again!")  # catch all for invalid inputs
        return stone_input(is_white)
    except ValueError:
        if options == True:
            print()
        else:
            print("Invalid input, try again!")  # catch all for invalid inputs
        return stone_input(is_white)



def place_stone(row, column, is_white):
    '''places stone according to offered coordinates and color'''
    if is_white:
        board[row - 1][column - 1] = 'O  '  # white placement
    else:
        board[row - 1][column - 1] = 'X  '  # black placement



def valid_stone(row, column):
    '''takes input from place_stone to determine if a stone may be placed'''
    if not 0 < row < 20 and 0 < column < 20:
        print("That placement is out of bounds, please re-enter your desired row
and column\n")
    else:
        if "-" in board[row - 1][column - 1]:  # empty space checker
            return True
        print("That tile is occupied by a piece, please re-enter your desired row
and column\n")
    return False



white = False
is_winner = False
white_pts = 0
black_pts = 0
write_board()
while not not_quitting:
    while not is_winner:
        white = not white
        row, column = stone_input(white)
        stats = check_adjacent_top([row, column])
        print_board()
        is_winner = stats[1]
```

```
    if white:
        white_pts += stats[0]
    else:
        black_pts += stats[0]
    if black_pts >= 10 or white_pts >= 10:
        is_winner = True
    if is_winner:
        numGames += 1
        if white:
            # call winner method
            numWhiteWins += 1
            writeData(numGames, True, black_pts, white_pts)
            print("\n\n\nCheck out your victory screen, White!\n\n\n")
            print("DO NOT INPUT ANYTHING UNTIL VICTORY SCREEN IS FINISHED\n")
            turtleWhiteWin()
        else:
            # call black winner method here
            numBlackWins += 1
            writeData(numGames, False, black_pts, white_pts)
            print("\n\n\nCheck out your victory screen, Black!\n\n\n")
            print("DO NOT INPUT ANYTHING UNTIL VICTORY SCREEN IS FINISHED\n")
            turtleBlackWin()
is_winner = False
white = False
clear_board()
```